Ders 13 Çalışma Özeti

Ders 13 Çalışma Özeti

Genel Konular

- İkili yığın ağacı (binary heap tree): max heap ve min heap.
- Heapify işlemi ve karmaşıklığı.
- Rastgele bir ağacı (diziyi) max heap'e dönüştürme (build max heap).
- Heap'in avantajları, dezavantajları ve kullanım alanları.
- Heap'e ekleme (insertion, up-heap) ve heap'ten silme (deletion, down-heap).
- Yığın sıralaması (heap sort) ve karmaşıklığı.
- Alıştırma problemleri: heap kontrolü, k'ıncı en küçük/büyük eleman, min heap'i max heap'e çevirme.

Hocanın Özellikle Vurguladığı Kısımlar

- Bir ağacın heap olması için iki koşul vardır:
  - Complete binary tree olmalıdır.
  - Max heap'te her ebeveyn çocuklarından büyük, min heap'te her ebeveyn çocuklarından küçük olmalıdır.
- Kardeşler arasında bir sıra kuralı yoktur. Sol çocuğun sağdan büyük ya da küçük olması önemli değildir.
- Ekleme ve silme ağacın yapısını bozabilir. Bu yüzden her değişiklikten sonra heapify yapılır.
- Ağaç complete olduğu için tek tarafa yaslanamaz. Bu yüzden ekleme ve silme en kötü durumda da $O(\log n)$'dir.
- Heap, arama (search) için uygun değildir. Bir değeri aramak için tüm ağaç gezilir.
- Heap'i yazdırınca "neredeyse sıralı" görünür. "Tepede en büyük var, altı zaten sıralıdır" düşüncesi yanlıştır.
- Bir dizinin heap olup olmadığı sorulduğunda hem max hem min heap kontrolü yapılmalıdır.
- Heap sort kararlı (stable) değildir.

Kısa Tekrar Notları

- Bu derste dizi 1'den başlar: `i` düğümünün sol çocuğu `2i`, sağ çocuğu `2i + 1`, ebeveyni `i / 2`.
- Max heap'te kök en büyük, min heap'te kök en küçük elemandır.

İşlem	Karmaşıklık
Ekleme (insert)	$O(\log n)$
Silme (delete)	$O(\log n)$
Heapify	$O(\log n)$
Max heap'te en büyük / min heap'te en küçük (`peek`, `getMax`, `getMin`)	$O(1)$
Arama (search)	$O(n)$
Heap sort	$O(n \log n)$

- Max heap'te en küçük eleman bir yaprakta olur. Yalnız yapraklara bakmak yeter.

- Heap'in avantajları:
  - En büyük/en küçük elemana hızlı erişim.
  - Verimli ekleme ve silme.
  - Dizi ile kolay gerçekleme.
- Heap'in dezavantajları:
  - Arama için uygun değil.
  - Ek yükü (overhead) var.
  - Öncelik gerekmeyen kuyruklarda dizi ya da bağlı listeden yavaştır.
- Kullanım alanları:
  - Öncelikli kuyruk (priority queue) ve heap sort.
  - Dijkstra en kısa yol, Prim minimum yayılan ağaç (minimum spanning tree), A* arama.
  - Huffman kodlaması, dinamik programlama.
  - Hastane triyaj sistemleri, sunucu yük dengeleme (load balancing), finans uygulamaları.

Detaylı Açıklamalar

Heap Örnekleri

```
Max heap            Min heap
    6                   2
   / \                 /   \
  4   5               5     7
 / \  /              / \   / \
3  2 0              6   8 9   11
```

- Max heap'te 4 ve 5, 6'dan küçüktür; 3 ve 2, 4'ten küçüktür; 0, 5'ten küçüktür.
- Dizi gösteriminde derste `6, 4, 5, 3, 2, 0, 1` örneği de kullanıldı; bu dizi de max heap'tir.
- Derste min heap'teki 8 ile 11'in yeri değiştirildi. Ağaç yine min heap olur. Aynı seviyedeki düğümler arasında sıra şartı yoktur; önemli olan ebeveynin çocuklarından küçük olmasıdır.
- Son seviyesinde arada boşluk olan (sola yaslı olmayan) ağaç, değerleri ne olursa olsun heap değildir.

Heapify (Aşağı Yönlü)

Heapify, ağacı heap kurallarına uydurma işlemidir. Ayrıca bir diziden heap oluşturma işlemi için de kullanılır.

```
void maxHeapify(int A[], int i, int n) {
    int left = 2 * i, right = 2 * i + 1;
    int largest = i;
    if (left <= n && A[left] > A[i])
        largest = left;
    if (right <= n && A[right] > A[largest])
        largest = right;
    if (largest != i) {
        swap(&A[i], &A[largest]);
        maxHeapify(A, largest, n);
    }
}
```

- İkinci karşılaştırma `A[i]` ile değil, `A[largest]` ile yapılır. Böylece iki çocuktan büyük olan seçilir.
- `largest == i` ise düğüm doğru yerdedir. Özyineleme burada durur.
- Eleman her adımda bir seviye iner. En fazla ağacın yüksekliği kadar adım atılır: $O(\log n)$.

Örnek: `[9, 7, 8, 3, 2, 6, 5]` max heap'inde kökteki 9 yerine 4 konuyor.

Adım	Dizi	Açıklama
Başlangıç	4, 7, 8, 3, 2, 6, 5	Kök 4 oldu, kural bozuldu

Adım	Dizi	Açıklama
1	8, 7, 4, 3, 2, 6, 5	4'ün çocukları 7 ve 8; en büyük 8 yukarı çıktı
2	8, 7, 6, 3, 2, 4, 5	4'ün çocukları 6 ve 5; 6 yukarı çıktı, 4 yaprağa indi

- Kök 4 yerine 14 olsaydı hiçbir çocuk daha büyük olmazdı. Heapify ilk adımda biterdi.

Diziden Max Heap Oluşturma (Build Max Heap)

- Ağaç hakkında bilgi yoksa her iç düğüm için heapify yapılır.
- Yaprak olmayan son düğümden (`n / 2`) başlanır ve köke kadar geriye gidilir.

```
void buildMaxHeap(int A[], int n) {
    for (int i = n / 2; i >= 1; i--)
        maxHeapify(A, i, n);
}
```

Örnek 1: `[5, 7, 6, 4, 8, 9, 1]`

Tur	`i` (değer)	Sonuç
1	3 (6)	5, 7, 9, 4, 8, 6, 1
2	2 (7)	5, 8, 9, 4, 7, 6, 1
3	1 (5)	9, 8, 5, 4, 7, 6, 1, sonra 5 bir daha iner: 9, 8, 6, 4, 7, 5, 1

Örnek 2: `[14, 7, 12, 13, 17, 6, 4, 9, 15]`

Tur	`i` (değer)	Sonuç
1	4 (13)	15 yukarı: 14, 7, 12, 15, 17, 6, 4, 9, 13
2	3 (12)	Değişiklik yok
3	2 (7)	17 yukarı: 14, 17, 12, 15, 7, 6, 4, 9, 13
4	1 (14)	17 yukarı, sonra 15 yukarı: 17, 15, 12, 14, 7, 6, 4, 9, 13

Ekleme (Up-Heap)

- Yeni eleman, dizinin son elemanından sonraki ilk boş göze konur. Böylece ağaç complete kalır.
- Sonra eleman ebeveyniyle karşılaştırılır. Büyükse yer değiştirir; bu, köke ya da daha büyük bir ebeveyne kadar sürer.
- Aşağı yönlü heapify'dan farkı: çocuklar arasında karşılaştırma yapılmaz, yalnız ebeveynle karşılaştırılır.

Başlangıç	Eklenen	Sonuç
9, 6, 8, 2, 5, 7	10	10, 6, 9, 2, 5, 7, 8
9, 8, 6, 1, 2, 5	7	9, 8, 7, 1, 2, 5, 6
9, 8, 7, 1, 2, 5, 6	12	12, 9, 7, 8, 2, 5, 6, 1

Silme

- Silinecek eleman son elemanla yer değiştirir.
- Eleman sayısı `n` bir azaltılır. Silinen eleman böylece devre dışı kalır.

- Yerine gelen elemandan başlayarak heapify yapılır.
- Silme genelde kökteki elemana uygulanır. Amaç öncelikli kuyruktan en öncelikli elemanı çıkarmaktır. Teknik olarak herhangi bir eleman da silinebilir.

Örnek: `[9, 8, 6, 2, 3, 1]` heap'inden 9 siliniyor (ekrandaki değer altyazıda 5 diye geçiyor; adımlar 3 ile tutarlı).

Adım	Dizi
9 ile son eleman 1 yer değiştirir, `n` 5 olur	1, 8, 6, 2, 3
8 yukarı çıkar	8, 1, 6, 2, 3
3 yukarı çıkar, 1 yaprağa iner	8, 3, 6, 2, 1

Heap Sort

- Kökteki en büyük eleman, dizinin son elemanıyla yer değiştirir.
- `n` bir azaltılır; son göz artık sıralı kısımdır.
- Kökten heapify yapılır.
- İşlem tek eleman kalana kadar tekrarlanır.

Örnek: `[9, 6, 8, 2, 5, 7]` (`|` sağı sıralı kısım)

Tur	Dizi
1	9 ↔ 7, heapify: 8, 6, 7, 2, 5 | 9
2	8 ↔ 5, heapify: 7, 6, 5, 2 | 8, 9
3	7 ↔ 2, heapify: 6, 2, 5 | 7, 8, 9
4	6 ↔ 5: 5, 2 | 6, 7, 8, 9
5	5 ↔ 2: 2 | 5, 6, 7, 8, 9

Derste daha büyük bir örnek de çözüldü: `[19, 16, 18, 13, 12, 14, 15, 7, 6, 4, 5]`.

- 1. tur: 19 ile 5 yer değiştirir. 5 önce 18, sonra 15 ile yer değiştirir: `18, 16, 15, 13, 12, 14, 5, 7, 6, 4 | 19`.
- 2. tur: 18 ile 4 yer değiştirir. 4 sırayla 16, 13 ve 7 ile yer değiştirir: `16, 13, 15, 7, 12, 14, 5, 4, 6 | 18, 19`.
- Sonraki turlar aynı şekilde sürer. `n` küçüldükçe sıralı kısımdaki düğümler artık çocuk sayılmaz.

Karmaşıklık:

- Her tur bir heapify ($O(\log n)$) içerir. n tur vardır. Toplam $O(n \log n)$.
- Yalnız en büyük k eleman isteniyorsa k tur yeter; maliyet $O(k \log n)$ düzeyine iner.
- Heap sort kararlı değildir. Örneğin aynı adlı iki kaydın sıralama sonrası sırası korunmayabilir.
- Merge sort ve quicksort ile aynı mertebededir ama pratikte daha az tercih edilir.

Alıştırma Problemleri

Hoca bunları gelecek haftaya alıştırma olarak bıraktı:

- Verilen ikili ağacın heap olup olmadığını bulmak.
  - Önce complete binary tree olup olmadığı kontrol edilir.
  - Dizi gösteriminde 1'den n'e kadar boş göz olmamalıdır.
  - Bağlı yapıda nasıl yapılacağı soru olarak bırakıldı.
- Sırasız bir dizide k'ıncı en küçük ya da en büyük elemanı bulmak.
- Verilen bir min heap'i max heap'e dönüştürmek. Örnek min heap: 2, 3, 5, 9, 2, 8, 7.

Ders sonrası notlar

- **Bağlı yapıda complete kontrolü:** Ağaç seviye seviye (kuyruk ile, level order) gezilir. İlk eksik çocuk görüldükten sonra hiçbir düğümün çocuğu olmamalıdır. Başka bir yol: düğüm sayısı n sayılır, kökün indeksi 1 kabul edilip çocuklara `2i` ve `2i + 1` verilir; hiçbir indeks n'i geçmemelidir.
- **Min heap'ten max heap'e:** Diziyi doğrudan `buildMaxHeap` ile işlemek yeterlidir. Min heap olması bir avantaj sağlamaz; işlem $O(n)$'dir.
- **Build heap karmaşıklığı:** Derste her heapify $O(\log n)$ sayılarak sezgisel bir hesap yapıldı. Sıkı analizde `buildMaxHeap` $O(n)$'dir, çünkü düğümlerin çoğu alt seviyelerdedir ve az iner. Heap sort'un toplamı yine $O(n \log n)$'dir.
- **Max heap'te en küçük:** Hoca en küçüğün "son seviyede" olduğunu söyledi. Doğrusu: en küçük eleman bir yapraktadır. Complete ağaçta yapraklar son seviyede ve sondan bir önceki seviyenin sağ tarafında olabilir. Dizide `n/2 + 1`'den `n`'e kadar olan gözlere bakmak yeter.
- **Ortadan silme:** Kök dışında bir eleman silinirken yerine gelen son eleman, ebeveyninden büyük olabilir. Bu durumda aşağı değil yukarı yönlü düzeltme (up-heap) gerekir.

Not: İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.